



Mata Kuliah

2025

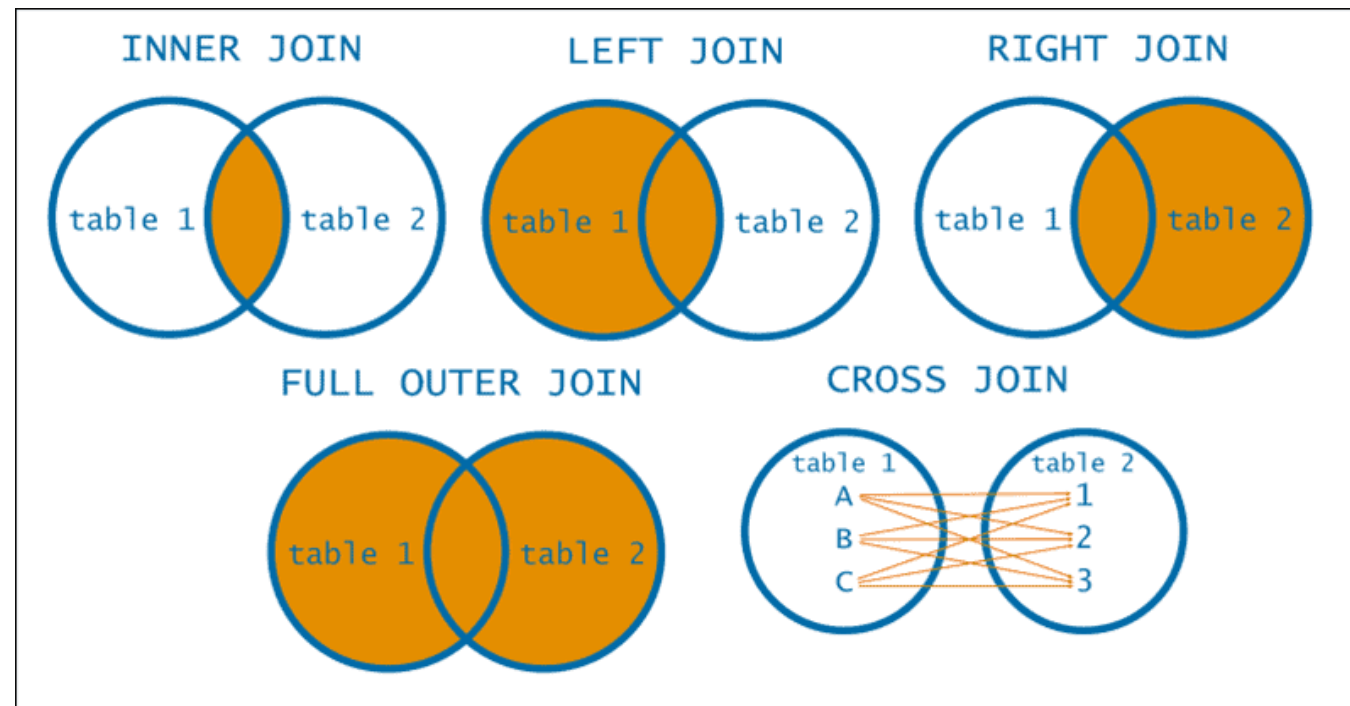
SISTEM BASIS DATA

Informatika Medis

DML - Join

DML – JOIN

JOIN digunakan untuk menggabungkan baris dari dua tabel atau lebih, berdasarkan kolom terkait di antara keduanya



DML – Inner JOIN

INNER JOIN memilih rekaman yang memiliki nilai yang cocok di kedua tabel dimana memiliki nilai penghubung yang sama.

```
SELECT column_name(s)
FROM table1
INNER JOIN table2
ON table1.column_name = table2.column_name;
```

Contoh:

```
SELECT Orders.OrderID, Customers.CustomerName
FROM Orders
INNER JOIN Customers ON Orders.CustomerID = Customers.CustomerID;
```

Apa Fungsi query dibawah? Ada yg bisa menjawab?

```
SELECT Orders.OrderID, Customers.CustomerName, Shippers.ShipperName
FROM ((Orders
INNER JOIN Customers ON Orders.CustomerID = Customers.CustomerID)
INNER JOIN Shippers ON Orders.ShipperID = Shippers.ShipperID);
```

DML – Left JOIN

LEFT JOIN mengembalikan semua rekaman dari tabel kiri (tabel1), dan rekaman yang cocok (jika ada) dari tabel kanan (tabel2).

```
SELECT column_name(s)
FROM table1
LEFT JOIN table2
ON table1.column_name = table2.column_name;
```

Contoh:

```
SELECT Customers.CustomerName, Orders.OrderID
FROM Customers
LEFT JOIN Orders ON Customers.CustomerID = Orders.CustomerID
ORDER BY Customers.CustomerName;
```

DML – Right JOIN

RIGHT JOIN mengembalikan semua rekaman dari tabel sebelah kanan (tabel2), dan rekaman yang cocok (jika ada) dari tabel sebelah kiri (tabel1).

```
SELECT column_name(s)
FROM table1
RIGHT JOIN table2
ON table1.column_name = table2.column_name;
```

Contoh:

```
SELECT Orders.OrderID, Employees.LastName, Employees.FirstName
FROM Orders
RIGHT JOIN Employees ON Orders.EmployeeID = Employees.EmployeeID
ORDER BY Orders.OrderID;
```

DML – Self JOIN

SELF Join merupakan gabungan biasa, tetapi tabel digabung dengan dirinya sendiri.

```
SELECT column_name(s)
FROM table1 T1, table1 T2
WHERE condition;
```

Contoh:

```
SELECT A.CustomerName AS CustomerName1, B.CustomerName AS CustomerName2, A.City
FROM Customers A, Customers B
WHERE A.CustomerID <> B.CustomerID
AND A.City = B.City
ORDER BY A.City;
```

DML – UNION

Operator **UNION** digunakan untuk menggabungkan kumpulan hasil dari dua **SELECT** pernyataan atau lebih.

- Setiap **SELECT** pernyataan di dalam **UNION** harus memiliki jumlah kolom yang sama
- Kolom juga harus memiliki tipe data yang serupa
- Kolom-kolom di setiap **SELECT** pernyataan juga harus dalam urutan yang sama

```
SELECT column_name(s) FROM table1  
UNION ALL  
SELECT column_name(s) FROM table2;
```

Contoh:

```
SELECT City FROM Customers  
UNION ALL  
SELECT City FROM Suppliers  
ORDER BY City;
```

DML – GROUP BY

GROUP BY mengelompokkan baris-baris yang memiliki nilai yang sama ke dalam baris-baris ringkasan, seperti "temukan jumlah pelanggan di setiap negara".

GROUP BY sering digunakan dengan fungsi agregat (**COUNT()**, **MAX()**, **MIN()**, **SUM()**, **AVG()**) untuk mengelompokkan kumpulan hasil berdasarkan satu atau lebih kolom.

```
SELECT column_name(s)
FROM table_name
WHERE condition
GROUP BY column_name(s)
ORDER BY column_name(s);
```

Contoh:

```
SELECT COUNT(CustomerID), Country
FROM Customers
GROUP BY Country
ORDER BY COUNT(CustomerID) DESC;
```


DML – HAVING

HAVING ditambahkan ke SQL karena **WHERE** tidak dapat digunakan dengan fungsi agregat.

```
SELECT column_name(s)
FROM table_name
WHERE condition
GROUP BY column_name(s)
HAVING condition
ORDER BY column_name(s);
```

Contoh:

```
SELECT Employees.LastName, COUNT(Orders.OrderID) AS NumberOfOrders
FROM (Orders
INNER JOIN Employees ON Orders.EmployeeID = Employees.EmployeeID)
GROUP BY LastName
HAVING COUNT(Orders.OrderID) > 10;
```

DML – EXISTS

Operator EXISTS digunakan untuk menguji keberadaan rekaman apa pun dalam subquery. Operator EXISTS mengembalikan TRUE jika subquery mengembalikan satu atau lebih rekaman.

```
SELECT column_name(s)
FROM table_name
WHERE EXISTS
(SELECT column_name FROM table_name WHERE condition);
```

Contoh:

```
SELECT SupplierName
FROM Suppliers
WHERE EXISTS (SELECT ProductName FROM Products WHERE Products.SupplierID =
Suppliers.supplierID AND Price = 22);
```

DML – ANY, ALL

Operator ANY dan ALL memungkinkan melakukan perbandingan antara nilai kolom tunggal dan rentang nilai lainnya.

ANY mengembalikan nilai boolean sebagai hasil mengembalikan TRUE jika **SALAH SATU** dari nilai subquery memenuhi kondisi

ANY berarti kondisi akan benar jika operasinya benar untuk nilai mana pun dalam rentang.

```
SELECT column_name(s)
FROM table_name
WHERE column_name operator ANY
  (SELECT column_name
   FROM table_name
   WHERE condition);
```

Contoh:

```
SELECT ProductName
FROM Products
WHERE ProductID = ANY
  (SELECT ProductID
   FROM OrderDetails
   WHERE Quantity = 10);
```

ALL mengembalikan nilai boolean sebagai hasil

Mengembalikan TRUE jika **SEMUA** nilai subquery memenuhi kondisi

Digunakan dengan pernyataan **SELECT**, **WHERE** dan **HAVING**

ALL berarti kondisi akan benar hanya jika operasinya benar untuk semua nilai dalam rentang.

```
SELECT column_name(s)
FROM table_name
WHERE column_name operator ALL
  (SELECT column_name
   FROM table_name
   WHERE condition);
```

Contoh:

```
SELECT ProductName
FROM Products
WHERE ProductID = ANY
  (SELECT ProductID
   FROM OrderDetails
   WHERE Quantity = 10);
```

DML – INSERT INTO SELECT

INSERT INTO SELECT menyalin data dari satu tabel dan memasukkannya ke tabel lain.

INSERT INTO SELECT mengharuskan tipe data dalam tabel sumber dan target cocok.

```
INSERT INTO table2 (column1, column2, column3, ...)
SELECT column1, column2, column3, ...
FROM table1
WHERE condition;
```

Contoh:

```
INSERT INTO Customers (CustomerName, City, Country)
SELECT SupplierName, City, Country FROM Suppliers
WHERE Country='Germany';
```

DML – CASE

CASE menelusuri kondisi dan mengembalikan nilai saat kondisi pertama terpenuhi (seperti pernyataan if-then-else). Jadi, setelah kondisi benar, pernyataan tersebut akan berhenti membaca dan mengembalikan hasilnya. Jika tidak ada kondisi yang benar, pernyataan tersebut mengembalikan nilai dalam ELSE.

Jika tidak ada ELSE dan tidak ada kondisi yang benar, maka akan mengembalikan NULL.

```
CASE
  WHEN condition1 THEN result1
  WHEN condition2 THEN result2
  WHEN conditionN THEN resultN
  ELSE result
END;
```

Contoh:

```
SELECT OrderID, Quantity,
CASE
  WHEN Quantity > 30 THEN 'The quantity is greater than 30'
  WHEN Quantity = 30 THEN 'The quantity is 30'
  ELSE 'The quantity is under 30'
END AS QuantityText
FROM OrderDetails;
```

Sumber

- Umar Zaky, S.Kom., M.Cs.
- W3schools
- phoenixnap.com



Terima Kasih